

SC2207 Database

Lab Project 4 & 5

REP2 Group 1:

Lim Zhi Yong

Hui Xujia

Ethan Lee Kei Shen

Huang Caihong

Chong Huai Zhi

Bhati Nancy

SQL DDL Commands for Table Creation

USER_ACCOUNT(UID, Gender, DOB, Name)

```
CREATE TABLE USER_ACCOUNT (  
    UID INT NOT NULL,  
    Gender TEXT NOT NULL,  
    DOB DATETIME NOT NULL,  
    Name TEXT NOT NULL,  
    PRIMARY KEY (UID)  
);
```

SHOP(SID, Type, MID, Shop_Manager_ID, Shop_Manager_Name)

```
CREATE TABLE SHOP(  
    SID INT NOT NULL,  
    Type TEXT NOT NULL,  
    MID INT NOT NULL,  
    Shop_Manager_ID INT NOT NULL,  
    Shop_Manager_Name TEXT NOT NULL,  
    FOREIGN KEY (MID) REFERENCES MALL(MID) ON DELETE CASCADE,  
    PRIMARY KEY(SID)  
);
```

SHOPPING(SID, UID, Amount_spent, Date_time_in, Date_time_out)

```
CREATE TABLE SHOPPING (  
    SID INT NOT NULL,  
    UID INT NOT NULL,  
    Amount_spent FLOAT NOT NULL,  
    Date_time_in DATETIME NOT NULL,  
    Date_time_out DATETIME NOT NULL,  
    PRIMARY KEY (SID, UID, Date_time_in),  
    FOREIGN KEY (SID) REFERENCES SHOP(SID) ON DELETE CASCADE,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE  
);
```

DINING(UID, OID, Amount_spent, Date_time_in, Date_time_out)

```
CREATE TABLE DINING (  
    UID INT NOT NULL,  
    OID INT NOT NULL,  
    Amount_spent FLOAT NOT NULL,  
    Date_time_in DATETIME NOT NULL,
```

```
Date_time_out DATETIME NOT NULL,  
PRIMARY KEY (UID, OID, Date_time_in),  
FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE,  
FOREIGN KEY (OID) REFERENCES RESTAURANT_OUTLET(OID) ON DELETE CASCADE  
);
```

MALL(MID, Address, NumShops, CID)

```
CREATE TABLE MALL (  
    MID INT NOT NULL,  
    Address NVARCHAR(255)TEXT NOT NULL,  
    NumShops INT NOT NULL,  
    CID INT NOT NULL,  
    FOREIGN KEY (CID) REFERENCES MALL_MGMT_COMPANY(CID) ON DELETE CASCADE,  
    PRIMARY KEY(MID)  
);
```

MALL_MGMT_COMPANY(CID, Address)

```
CREATE TABLE MALL_MGMT_COMPANY (  
    CID INT NOT NULL ,  
    Address TEXT NOT NULL,  
    PRIMARY KEY(CID)  
);
```

RESTAURANT_OUTLET(OID, Unit_no, MID, RID, Type, Name, Floor)

```
CREATE TABLE RESTAURANT_OUTLET (  
    OID INT NOT NULL,  
    Unit_no TEXT NOT NULL,  
    MID INT NOT NULL,  
    RID INT NOT NULL,  
    Type TEXT NOT NULL,  
    Name NVARCHAR(255) NOT NULL,  
    Floor INT NOT NULL,  
    FOREIGN KEY (MID) REFERENCES MALL(MID) ON DELETE CASCADE,  
    FOREIGN KEY (RID) REFERENCES RESTAURANT_CHAIN(RID) ON DELETE CASCADE,  
    PRIMARY KEY(OID)  
);
```

RESTAURANT_CHAIN(RID, Address)

```
CREATE TABLE RESTAURANT_CHAIN (  

```

```
RID INT NOT NULL,  
Address TEXT NOT NULL,  
PRIMARY KEY(RID)  
);
```

DAY_PACKAGE (DID, Description, VID, UID, Date, ProductID)

```
CREATE TABLE DAY_PACKAGE (  
    DID INT NOT NULL,  
    Description TEXT,  
    VID INT NOT NULL,  
    UID INT NOT NULL,  
    Date DATETIME NOT NULL,  
    ProductID INT NOT NULL,  
    FOREIGN KEY (VID) REFERENCES VOUCHER(VID),  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID),  
    PRIMARY KEY(DID)  
);
```

COMPLAINT (CID,Text,Status,Filed_date_time,UID)

```
CREATE TABLE COMPLAINT (  
    CID INT NOT NULL,  
    Text TEXT NOT NULL,  
    Status TEXT NOT NULL,  
    Filed_date_time DATETIME NOT NULL,  
    UID INT NOT NULL,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID),  
    PRIMARY KEY(CID)  
);
```

COMPLAINTS_ON_RESTAURANT (CID, OID)

```
CREATE TABLE COMPLAINTS_ON_RESTAURANT (  
    CID INT NOT NULL,  
    OID INT NOT NULL,  
    PRIMARY KEY (CID, OID),  
    FOREIGN KEY (CID) REFERENCES COMPLAINT(CID),  
    FOREIGN KEY (OID) REFERENCES RESTAURANT_OUTLET(OID)  
);
```


COMPLAINTS_ON_SHOP(CID, SID)

```
CREATE TABLE COMPLAINTS_ON_SHOP (  
    CID INT NOT NULL,  
    SID INT NOT NULL,  
    PRIMARY KEY (CID, SID),  
    FOREIGN KEY (CID) REFERENCES COMPLAINT(CID),  
    FOREIGN KEY (SID) REFERENCES SHOP(SID)  
);
```

VOUCHER(VID, Date_issued, Description, Status, Expiry_date)

```
CREATE TABLE VOUCHER (  
    VID INT NOT NULL,  
    Date_issued DATETIME NOT NULL,  
    Description TEXT,  
    Status TEXT NOT NULL,  
    Expiry_date DATETIME NOT NULL,  
    PRIMARY KEY(VID)  
);
```

PURCHASE_VOUCHER(VID, Purchase_discount, UID, Date_time)

```
CREATE TABLE PURCHASE_VOUCHER (  
    VID INT NOT NULL,  
    Purchase_discount FLOAT NOT NULL,  
    UID INT NOT NULL,  
    Date_time DATETIME NOT NULL,  
    FOREIGN KEY (VID) REFERENCES VOUCHER(VID) ON DELETE CASCADE,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE,  
    PRIMARY KEY(VID)  
);
```

DINE_VOUCHER(VID, Cash_discount, UID, Date_time)

```
CREATE TABLE DINE_VOUCHER (  
    VID INT NOT NULL,  
    Cash_discount FLOAT NOT NULL,  
    UID INT NOT NULL,  
    Date_time DATETIME NOT NULL,  
    FOREIGN KEY (VID) REFERENCES VOUCHER(VID) ON DELETE CASCADE,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE,  
    PRIMARY KEY(VID)  
);
```

GROUP_VOUCHER(VID, Group_size, Group_discount, UID, Date_time)

```
CREATE TABLE GROUP_VOUCHER (  
    VID INT NOT NULL,  
    Group_size INT NOT NULL,  
    Group_discount FLOAT NOT NULL,  
    UID INT NOT NULL,  
    Date_time DATETIME NOT NULL,  
    FOREIGN KEY (VID) REFERENCES VOUCHER(VID) ON DELETE CASCADE,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE,  
    PRIMARY KEY(VID)  
);
```

PACKAGE_VOUCHER(VID, Package_discount, UID, Date_time)

```
CREATE TABLE PACKAGE_VOUCHER (  
    VID INT NOT NULL,  
    Package_discount FLOAT NOT NULL,  
    UID INT NOT NULL,  
    Date_time DATETIME NOT NULL,  
    FOREIGN KEY (VID) REFERENCES VOUCHER(VID) ON DELETE CASCADE,  
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID) ON DELETE CASCADE,  
    PRIMARY KEY(VID)  
);
```

RECOMMENDATION(NID, Valid_period, Date_issued, MID, Dine_voucher, Purchase_voucher, DID, OID)

```
CREATE TABLE RECOMMENDATION (  
    NID INT NOT NULL,  
    Valid_period INT NOT NULL,  
    Date_issued DATETIME NOT NULL,  
    MID INT NOT NULL,  
    Dine_voucher INT NOT NULL,  
    Purchase_voucher INT NOT NULL,  
    DID INT NOT NULL,  
    OID INT NOT NULL,  
    FOREIGN KEY (MID) REFERENCES MALL(MID),  
    FOREIGN KEY (Dine_voucher) REFERENCES DINE_VOUCHER(VID),  
    FOREIGN KEY (Purchase_voucher) REFERENCES PURCHASE_VOUCHER(VID),  
    FOREIGN KEY (DID) REFERENCES DAY_PACKAGE(DID),  
    FOREIGN KEY (OID) REFERENCES RESTAURANT_OUTLET(OID),
```

```
PRIMARY KEY(NID)
);
```

USER_RELATED_USER(UID1, UID2, Type, GID)

```
CREATE TABLE USER_RELATED_USER (
    UID1 INT NOT NULL,
    UID2 INT NOT NULL,
    Type TEXT NOT NULL,
    GID INT NOT NULL,
    PRIMARY KEY (UID1, UID2),
    FOREIGN KEY (UID1) REFERENCES USER_ACCOUNT(UID),
    FOREIGN KEY (UID2) REFERENCES USER_ACCOUNT(UID)
);
```

USER_USE_RECOMMENDATION(UID, NID)

```
CREATE TABLE USER_USE_RECOMMENDATION (
    UID INT NOT NULL,
    NID INT NOT NULL,
    PRIMARY KEY (UID, NID),
    FOREIGN KEY (UID) REFERENCES USER_ACCOUNT(UID),
    FOREIGN KEY (NID) REFERENCES RECOMMENDATION(NID)
);
```

DAY_PACKAGE_HAS_MALL(DID, MID)

```
CREATE TABLE DAY_PACKAGE_HAS_MALL (
    DID INT NOT NULL,
    MID INT NOT NULL,
    PRIMARY KEY (DID, MID),
    FOREIGN KEY (DID) REFERENCES DAY_PACKAGE(DID),
    FOREIGN KEY (MID) REFERENCES MALL(MID)
);
```

DAY_PACKAGE_HAS_RESTAURANT_OUTLET(DID, OID)

```
CREATE TABLE DAY_PACKAGE_HAS_RESTAURANT_OUTLET (  
    DID INT NOT NULL,  
    OID INT NOT NULL,  
    PRIMARY KEY (DID, OID),  
    FOREIGN KEY (DID) REFERENCES DAY_PACKAGE(DID),  
    FOREIGN KEY (OID) REFERENCES RESTAURANT_OUTLET(OID)  
);
```

SQL Statements to Solve the Queries in Appendix B

1. Find the most popular day packages(product/service) where all participants are related to one another as either family members or members of the same club.

```
SELECT productid, count(productid) AS COUNT
FROM day_package dpb
WHERE (dpb.did IN
      (SELECT did
       FROM day_package dp0
       WHERE productid IN
        (SELECT dp1.productid
         FROM day_package dp1
         GROUP BY dp1.productid
         EXCEPT
        (SELECT dp2.productid
         FROM day_package dp2
         JOIN day_package dp3 ON dp2.date = dp3.date
         AND dp2.productid = dp3.productid
         AND dp2.uid <> dp3.uid
         JOIN user_related_user uru ON ((uru.uid1 = dp2.uid
                                         AND uru.uid2 = dp3.uid)
                                         OR (uru.uid2 = dp2.uid
                                              AND uru.uid1 = dp3.uid))
        WHERE cast(uru.type AS varchar(255)) NOT IN ('family', 'club')
        GROUP BY dp2.productid)))
      AND (did NOT IN
      (SELECT dp4.did
       FROM day_package dp4
       EXCEPT
      (SELECT dp2.did
       FROM day_package dp2
       JOIN day_package dp3 ON dp2.[date] = dp3.[date]
       AND dp2.productid = dp3.productid
       AND dp2.uid <> dp3.uid
       GROUP BY dp2.did)))
GROUP BY productid
ORDER BY count(productid) DESC;
```

Query Output:

Results			Messages	
	productid	COUNT		
1	100	8		
2	110	4		
3	103	3		

Brief explanation:

1. The main idea is to first, find all the day packages and MINUS (EXCEPT) the day packages whereby some participants are neither 'family' nor 'club'. The

results of this subquery would be the ProductID of the day packages whereby all participants are either 'family' or 'club'.

2. We then count the number of tuples in DAY_PACKAGE whereby ProductID is equal to the ones found in (1), but exclude those day packages whereby there are only one person in it, and order them in descending order. Although the question is asking for the "MOST", we did not use the TOP 1 keyword because we wanted our query to show 3-5 tuples as output.

Assumptions made:

1. Popularity is based on how many people signed up for the day package.
2. Tuples in the DAY_PACKAGE table with the same ProductID, will mean the same day package. However, they may have different dates.
3. Different dates would represent different "groups", one day package may have numerous groups.
4. There can only be 1 group going for the day package on each calendar date.
5. As long as everyone in the group is either 'family' or 'club', it will add to the popularity count.
6. For groups with only 1 person in it, we will exclude it from the count.

2. Find families who frequently shopped and dined together, with or without day packages. As part of your output, indicate whether these families use day packages or not. “frequently” means at least 50% of the time.

```
SELECT total_count_go_out.uid,
       us.name,
       total_count_go_out.count_together,
       total_count_go_out.count_total,
       total_count_go_out.percentage,
       total_count_dp.count_dp
FROM
  (SELECT count_all_go_out.uid,
         count_total,
         count_together,
         cast(sq10.count_together AS FLOAT) / cast(count_all_go_out.count_total AS
FLOAT) AS percentage
  FROM
    (SELECT UID,
           sum(all_count) AS count_total
     FROM (
       (SELECT UID,
              count(*) AS all_count
        FROM shopping
        GROUP BY UID)
      UNION
      (SELECT UID,
              count(*) AS all_count
        FROM dining
        GROUP BY UID)) AS ss1
     GROUP BY UID) AS count_all_go_out
  LEFT OUTER JOIN
    (SELECT UID,
           sum(count_together) AS count_together
     FROM (
       (SELECT UID,
              count(date_time_in) AS count_together
        FROM
          (SELECT DISTINCT d1.*
           FROM dining d1
           JOIN dining d2 ON d1.uid <> d2.uid
           AND d1.oid = d2.oid
           AND d1.date_time_in = d2.date_time_in
           JOIN (
             (SELECT uru1.uid2,
                    uru1.gid
              FROM user_related_user uru1
              JOIN user_related_user uru2 ON uru1.gid = uru2.gid
              WHERE cast(uru1.type AS varchar(255)) IN ('family'))
            UNION
            (SELECT uru1.uid1,
```

```

        uru1.gid
        FROM user_related_user uru1
        JOIN user_related_user uru2 ON uru1.gid = uru2.gid
        WHERE cast(uru2.type AS varchar(255)) IN ('family')) AS
sq1 ON d1.uid = sq1.uid2
        JOIN (
            (SELECT uru1.uid2,
                uru1.gid
            FROM user_related_user uru1
            JOIN user_related_user uru2 ON uru1.gid = uru2.gid
            WHERE cast(uru1.type AS varchar(255)) IN ('family'))
        UNION
            (SELECT uru1.uid1,
                uru1.gid
            FROM user_related_user uru1
            JOIN user_related_user uru2 ON uru1.gid = uru2.gid
            WHERE cast(uru2.type AS varchar(255)) IN ('family')) AS
sq2 ON d2.uid = sq2.uid2
        WHERE sq1.gid = sq2.gid ) AS ss1
    GROUP BY ss1.uid)
UNION ALL
    (SELECT ss2.uid,
        count(*) AS count_together
    FROM
        (SELECT DISTINCT s1.uid
        FROM shopping s1
        JOIN shopping s2 ON s1.uid <> s2.uid
        AND s1.sid = s2.sid
        AND s1.date_time_in = s2.date_time_in
        JOIN (
            (SELECT uru1.uid2,
                uru1.gid
            FROM user_related_user uru1
            JOIN user_related_user uru2 ON uru1.gid = uru2.gid
            WHERE cast(uru1.type AS varchar(255)) IN ('family'))
        UNION
            (SELECT uru1.uid1,
                uru1.gid
            FROM user_related_user uru1
            JOIN user_related_user uru2 ON uru1.gid = uru2.gid
            WHERE cast(uru2.type AS varchar(255)) IN ('family')) AS
sq1 ON s1.uid = sq1.uid2
        JOIN (
            (SELECT uru1.uid2,
                uru1.gid
            FROM user_related_user uru1
            JOIN user_related_user uru2 ON uru1.gid = uru2.gid
            WHERE cast(uru1.type AS varchar(255)) IN ('family'))
        UNION
            (SELECT uru1.uid1,
                uru1.gid

```



```

FROM user_related_user uru1
JOIN user_related_user uru2 ON uru1.gid = uru2.gid
WHERE cast(uru2.type AS varchar(255)) IN ('family')) AS
sq2 ON s2.uid = sq2.uid2
WHERE sq1.gid = sq2.gid) AS ss2
GROUP BY ss2.uid)) AS sss1
GROUP BY UID) AS sq10 ON sq10.uid = count_all_go_out.uid) total_count_go_out
LEFT OUTER JOIN
(SELECT UID,
sum(dp_count) AS count_dp
FROM (
(SELECT si1.uid,
count(*) AS dp_count
FROM
(SELECT dg.*
FROM dining dg
JOIN day_package dp ON dg.uid = dp.uid
AND convert(date, dg.[date_time_in]) = convert(date, dp.[date])) AS
si1
GROUP BY si1.uid)
UNION ALL
(SELECT si2.uid,
count(*) AS dp_count
FROM
(SELECT sp.*
FROM shopping sp
JOIN day_package dp ON sp.uid = dp.uid
AND convert(date, sp.[date_time_in]) = convert(date, dp.[date])) AS
si2
GROUP BY si2.uid)) AS ss1
GROUP BY UID) AS total_count_dp
ON total_count_go_out.uid = total_count_dp .uid
JOIN USER_ACCOUNT us ON us.UID = total_count_go_out.uid
WHERE total_count_go_out.percentage > 0.5;

```

Query Output:

100 %						
Results Messages						
	uid	name	count_together	count_total	percentage	count_dp
1	5	Christopher Wilson	3	3	1	3
2	6	Sarah Martinez	1	1	1	1

Brief explanation:

(We are focusing on user 5, 6 and 7 for this example.)

User 5, 6 and 7 belong to the same family.

User 5 dines together with user 7 on 31 March 2024.

User 5 shops together with user 6 on 31 March 2024.

User 5 shops together with user 7 on 31 March 2024.

User 7 shops alone 3 times.

Therefore we should expect the results to return user 5 and user 6 only.

1. Count_total represents the total number of shopping/dining activities done by the user (be it alone, or with family). Count_together represents the total number of shopping/dining activities done by the user WITH family. Percentage is Count_together DIVIDED by Count_total.
2. We first have to JOIN the SHOPPING table with USER_RELATED_USER table to find out if users are shopping with their family members.
3. We repeat (1) for the DINING table as well.
4. Afterwards, we sum up the frequencies whereby each user dines/shops together with their family.
5. We then join the resulting table with another table where we summed up the total number of times each user dined/shopped (either alone or with a family).
6. By dividing (3) with (4), we are able to filter out the users who shop/dine with their families more than 50% of the time.
7. We join the above with the DAY_PACKAGE table again, so that for each user that dines/shops with their family more than 50% of the time, we find out if they are using day package (when count_dp is greater than 0, it mean day package is used).

Assumptions:

1. Since one family has multiple members, and each member may either join the family, or shop/dine alone, it would be more appropriate to return the users within the families rather than the families itself.

3. What are the most popular recommendations from the app regarding malls?

```
SELECT [mid],  
       count(*) AS recommendation_count  
FROM recommendation  
GROUP BY [mid]  
ORDER BY recommendation_count DESC;
```

Query Output:

	MID	Recommendation_Count
1	1	3
2	2	2
3	3	1

Brief explanation:

1. GROUP BY MID means that the count of recommendations will be calculated for each unique Mall ID (MID).
2. After grouping them based on MID, we will count the number of tuples within each MID, this will give us the number of recommendations for each MID.
3. ORDER BY sorts the results based on the 'Recommendation_Count' in descending order with the highest number of recommendations appearing at the top of the result set.

4. Compulsive shoppers are those who have visited a certain mall more than 5 times within a certain period of time. Find the youngest compulsive shoppers and the amount they spent in total during December 2023.

```
SELECT TOP 1 U.UID,
            U.Name,
            U.DOB,
            Subquery2.TOTAL_SPENT,
            Subquery2.SHOPPING_COUNT,
            Subquery2.MID
FROM
    (SELECT *
    FROM
        (SELECT SP.UID,
                S.MID,
                COUNT(S.MID) AS SHOPPING_COUNT,
                SUM(SP.Amount_spent) AS TOTAL_SPENT
        FROM SHOPPING SP
        JOIN SHOP S ON SP.SID = S.SID
        WHERE SP.Date_time_in BETWEEN '2023-12-01' AND '2023-12-31'
        GROUP BY SP.UID,
                S.MID) AS Subquery1
        WHERE SHOPPING_COUNT > 4 ) AS Subquery2
JOIN USER_ACCOUNT U ON Subquery2.UID = U.UID
ORDER BY U.DOB DESC;
```

Query Output:

Results Messages					
UID	Name	DOB	TOTAL_SPENT	SHOPPING_COUNT	MID
1	John Smith	1990-05-15 00:00:00.000	2826.39	5	1

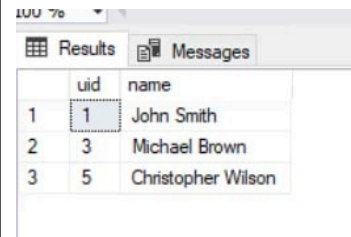
Brief explanation:

1. The first subquery (Subquery1) involves joining SHOP and SHOPPING so that we will be able to get the MID. Using the WHERE clause, we filtered it down by the date and GROUP BY UID and MID so that each tuple will represent the number of times a user visits a mall within the given period and the SUM function will give us the total amount spent by the user.
2. The second subquery (Subquery2) uses another WHERE clause to filter out only those tuples in Subquery1 that have a count of more than 4.
3. On the outermost query, we JOIN Subquery2 with USER_ACCOUNT table so that we have access to the DOB of the user. By setting ORDER BY U.DOB DESC, we are able to obtain the YOUNGEST shopper that has shopped more than 5 times at a mall during the given period of time.

5. Find users who have dined in all the restaurants in some malls, but have never dined in any restaurants in some other malls.

```
SELECT us.uid,
       [name]
FROM user_account us
JOIN
  (SELECT [uid]
   FROM
     (SELECT distinct(sq3.uid)
      FROM
        (SELECT sq1.uid,
                 sq1.mid,
                 count(DISTINCT sq1.oid) AS dine_in_same_mall
         FROM
           (SELECT dg.uid,
                    dg.oid,
                    ro.mid
            FROM dining AS dg
            JOIN restaurant_outlet AS ro ON dg.oid = ro.oid) AS sq1
         GROUP BY sq1.uid,
                  sq1.mid) AS sq3
      JOIN
        (SELECT ro.mid,
                 count(ro.oid) AS number_of_restaurants
         FROM restaurant_outlet ro
         GROUP BY ro.mid) AS sq2 ON sq3.mid = sq2.mid
      AND sq2.number_of_restaurants = sq3.dine_in_same_mall
      GROUP BY sq3.uid) AS s1
   EXCEPT SELECT distinct(sq13.uid)
  FROM
    (SELECT sq11.uid,
            sq11.mid,
            count(DISTINCT sq11.oid) AS dine_in_same_mall
     FROM
       (SELECT dg.uid,
                dg.oid,
                ro.mid
        FROM dining AS dg
        JOIN restaurant_outlet AS ro ON dg.oid = ro.oid) AS sq11
     GROUP BY sq11.uid,
              sq11.mid) AS sq13
  JOIN
    (SELECT ro.mid,
            count(ro.oid) AS number_of_restaurants
     FROM restaurant_outlet ro
     GROUP BY ro.mid) AS sq12 ON sq13.mid = sq12.mid
  AND sq12.number_of_restaurants <> sq13.dine_in_same_mall
  GROUP BY sq13.uid) AS s2 ON us.uid = s2.uid;
```

Query Output:



	uid	name
1	1	John Smith
2	3	Michael Brown
3	5	Christopher Wilson

Brief explanation:

(How we insert tuples into the database)

MallA has Restaurant 1 and 2

MallB has Restaurant 3 and 4

MallC has Restaurant 5 and 6

MallD has Restaurant 7 and 8

User1 dined in Restaurants 1, 2, 3, 4, 5 and 6.

User2 dined in Restaurants 1, 2, 3, 4 and 5.

User3 dined in Restaurants 1, 2, 1, 2, 1 and 2 again.

In this example, User1 and User3 should be the results of the query, but not User2 because he dined in Restaurant5 and not Restaurant6.

1. The main idea is to find UUIDs that have dined in every restaurant in the mall and MINUS (EXCEPT) UUIDs that have dined in less than the number of restaurants in the mall.
2. To achieve the above, we first JOIN DINING with RESTAURANT_OUTLET so we have access to the MID. This MID will then be used to JOIN with RESTAURANT_OUTLET again, but this time it has computed the number of restaurants for each MID using the GROUP BY MID and COUNT function.
3. The result of the above is essentially the DINING table with the addition of the corresponding number of restaurants in the mall the restaurant belongs to.
4. We then GROUP BY UUID and MID, such that we will be able to find the number of DISTINCT restaurants each user has dined in each mall, and check if that is EQUAL to the number of restaurants in the same mall. This gives us the first part mentioned in (1).
5. We repeat steps 2-4, except this time we find the UUIDs whereby the number of DISTINCT restaurants dined in each mall is NOT EQUAL to the number of restaurants in the same mall. This gives us the second part mentioned in (2).
6. We do an EXCEPT between (4) and (5) to obtain the UUIDs of those that have dined in all the restaurants in some malls, but have never dined in any restaurants in another mall.
7. The outermost query is simply to JOIN the immediate subquery with the USER_ACCOUNT table so that we have access to the user's name.

6. What are the top 3 highest earning malls and restaurants?

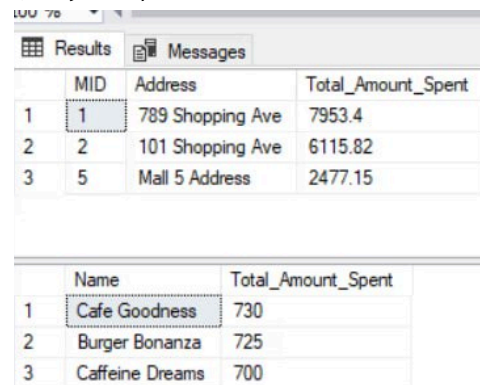
Top 3 highest earning malls

```
SELECT top 3 m.mid,  
            m.address,  
            sum(sp.amount_spent) AS total_amount_spent  
FROM mall m  
JOIN shop sh ON m.mid = sh.mid  
JOIN shopping sp ON sh.sid = sp.sid  
GROUP BY m.mid,  
         m.address  
ORDER BY total_amount_spent DESC;
```

Top 3 highest earning restaurants

```
SELECT top 3 r.name,  
            sum(d.amount_spent) AS total_amount_spent  
FROM restaurant_outlet r  
JOIN dining d ON r.oid = d.oid  
GROUP BY r.name  
ORDER BY total_amount_spent DESC;
```

Query Output:



	MID	Address	Total_Amount_Spent
1	1	789 Shopping Ave	7953.4
2	2	101 Shopping Ave	6115.82
3	5	Mall 5 Address	2477.15

	Name	Total_Amount_Spent
1	Cafe Goodness	730
2	Burger Bonanza	725
3	Caffeine Dreams	700

Brief explanation:

Malls

1. We join SHOPPING with SHOP so that we are able to identify which mall the shopping has taken place in, then we JOIN it with MALL so that we are able to get the Address of the mall. We then use GROUP BY MID and Address and SUM function to find how much money is spent shopping in each MID.
2. With the TOP 3 and ORDER BY DESC keywords, we are able to find the top 3 highest earning malls.

Restaurants

1. We join DINING with RESTAURANT_OUTLET so that we are able to get the Name of the restaurant. We then use GROUP BY Name and SUM function to find how much money is spent dining in each restaurant.

2. With the TOP 3 and ORDER BY DESC keywords, we are able to find the top 3 highest earning restaurants.

SQL Statements to Solve the Additional Queries given in class

7. Find restaurants that are more popular for young adults (18 to 35 years old) than for older adults (50 years or older).

```
SELECT SQ.OID,
       RO.Name,
       SQ.YOUNG_ADULTS,
       SQ.OLD_ADULTS
FROM
  (SELECT COALESCE(YOUNGQUERY.OID, OLDQUERY.OID) AS OID,
         COALESCE(YOUNG_ADULTS, 0) AS YOUNG_ADULTS,
         COALESCE(OLD_ADULTS, 0) AS OLD_ADULTS
   FROM
     (SELECT d.OID,
            COUNT(*) AS YOUNG_ADULTS
      FROM DINING D
      JOIN USER_ACCOUNT UA ON D.UID = UA.UID
      WHERE DATEDIFF(YEAR, UA.DOB, GETDATE()) BETWEEN 18 AND 35
      GROUP BY d.OID) AS YOUNGQUERY
    FULL OUTER JOIN
     (SELECT d.OID,
            COUNT(*) AS OLD_ADULTS
      FROM DINING D
      JOIN USER_ACCOUNT UA ON D.UID = UA.UID
      WHERE DATEDIFF(YEAR, UA.DOB, GETDATE()) >= 50
      GROUP BY d.OID) OLDQUERY ON YOUNGQUERY.OID = OLDQUERY.OID) AS SQ
JOIN RESTAURANT_OUTLET RO ON SQ.OID = RO.OID
WHERE SQ.YOUNG_ADULTS > SQ.OLD_ADULTS
ORDER BY SQ.OLD_ADULTS DESC;
```

Query Output:

	OID	Name	YOUNG_ADULTS	OLD_ADULTS
1	1	Burger Bonanza	2	0
2	2	Cafe Goodness	1	0

Brief explanation:

1. In subquery YOUNGQUERY, we find the count of diners who are aged 18 to 35, and group them based on the restaurant's OID.
2. In subquery OLDQUERY, we find the count of diners who are aged 50 and above, and group them based on the restaurant's OID.
3. We then do a FULL OUTER JOIN between these 2 queries, and assign all NULL values as 0 so that a comparison can be done in the later steps. This joined query will be SQ.
4. SQ is then joined with RESTAURANT_OUTLET table so that we have access to the restaurant's names.
5. Lastly, we add in a WHERE clause whereby we filter the output based on the restaurants that have a higher YOUNG_ADULT count than the OLD_ADULT count.

Printout of all table records

USER_ACCOUNT(UID, Gender, DOB, Name)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* - ✕

```
Select * from USER_ACCOUNT;
```

100 %

Results Messages

	UID	Gender	DOB	Name
1	1	Male	1990-05-15 00:00:00.000	John Smith
2	2	Female	1985-10-25 00:00:00.000	Emily Johnson
3	3	Male	1988-03-07 00:00:00.000	Michael Brown
4	4	Female	1993-12-18 00:00:00.000	Jessica Davis
5	5	Male	1982-08-30 00:00:00.000	Christopher Wilson
6	6	Female	1995-07-22 00:00:00.000	Sarah Martinez
7	7	Male	1979-11-09 00:00:00.000	Daniel Anderson

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* - ✕

```
Select * from USER_ACCOUNT;
```

100 %

Results Messages

	UID	Gender	DOB	Name
7	7	Male	1979-11-09 00:00:00.000	Daniel Anderson
8	8	Female	1987-04-12 00:00:00.000	Jennifer Taylor
9	9	Male	1991-09-03 00:00:00.000	Matthew Thomas
10	10	Female	1984-06-28 00:00:00.000	Laura Garcia
11	11	Male	1980-02-14 00:00:00.000	David Rodriguez
12	12	Female	1992-01-10 00:00:00.000	Ashley Hernandez
13	13	Male	1986-07-05 00:00:00.000	James Lopez
14	14	Female	1989-04-29 00:00:00.000	Mary Martinez
15	15	Male	1994-11-16 00:00:00.000	Joshua Gonzalez
16	16	Female	1983-08-08 00:00:00.000	Amanda Perez
17	17	Male	1981-03-21 00:00:00.000	Robert Sanchez
18	18	Female	1990-10-02 00:00:00.000	Melissa Lee
19	19	Male	1978-12-25 00:00:00.000	Christopher Young
20	20	Female	1996-09-19 00:00:00.000	Stephanie Scott
21	21	Male	1989-06-14 00:00:00.000	Justin King
22	22	Female	1982-01-03 00:00:00.000	Nicole Wright
23	23	Male	1993-04-26 00:00:00.000	Andrew Hill
24	24	Female	1980-11-11 00:00:00.000	Heather Flores
25	25	Male	1985-08-04 00:00:00.000	Brandon Green
26	26	Female	1991-05-27 00:00:00.000	Samantha Adams
27	27	Male	1979-02-18 00:00:00.000	Joseph Baker
28	28	Female	1986-07-12 00:00:00.000	Angela Mitchell
29	29	Male	1994-10-06 00:00:00.000	Tyler Carter
30	30	Female	1983-03-31 00:00:00.000	Rebecca Hall

SHOP(SID, Type, MID, Shop_Manager_ID, Shop_Manager_Name)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from SHOP
```

100 %

Results Messages

	SID	Type	MID	Shop_Manager_ID	Shop_Manager_Name
1	1	Clothing	1	1001	John Doe
2	2	Electronics	1	1002	Jane Smith
3	3	Grocery	2	1003	Michael Johnson
4	4	Footwear	2	1004	Emily Wilson
5	5	Books	3	1005	David Brown
6	6	Sporting Goods	3	1006	Sarah Davis
7	7	Jewelry	4	1007	James Taylor
8	8	Cosmetics	4	1008	Emma Martinez
9	9	Home Decor	5	1009	Olivia Anderson
10	10	Toys	5	1010	William Thomas
11	11	Pet Supplies	6	1011	Sophia White
12	12	Stationery	6	1012	Alexander Martinez
13	13	Furniture	7	1013	Ella Harris
14	14	Kitchenware	7	1014	Noah Clark
15	15	Music	8	1015	Ava Lewis
16	16	Health & Wellness	8	1016	Liam Robinson
17	17	Eyewear	9	1017	Mia Hall
18	18	Hardware	9	1018	Lucas King
19	19	Art Supplies	10	1019	Amelia Wright
20	20	Gifts & Novelties	10	1020	Benjamin Scott

SHOPPING(SID, UID, Amount_spent, Date_time_in, Date_time_out)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from SHOPPING;
```

100 %

	SID	UID	Amount_spent	Date_time_in	Date_time_out
1	1	1	812.2	2023-11-20 23:47:03.000	2023-11-21 00:42:23.000
2	1	1	305.96	2023-12-10 01:23:20.000	2023-12-10 10:21:50.000
3	1	1	469.64	2023-12-15 19:05:16.000	2023-12-15 22:28:47.000
4	1	1	962.31	2023-12-16 18:56:21.000	2023-12-16 20:07:22.000
5	1	1	836.54	2023-12-29 17:54:38.000	2023-12-29 23:41:22.000
6	1	1	150	2024-03-28 10:00:00.000	2024-03-28 11:00:00.000
7	1	5	10	2024-03-31 10:00:00.000	2024-03-31 11:00:00.000
8	1	7	12	2024-03-31 10:00:00.000	2024-03-31 11:00:00.000
9	1	10	28.32	2023-12-08 18:31:33.000	2023-12-08 22:12:45.000
10	1	11	236.35	2023-12-12 23:57:11.000	2023-12-13 00:30:38.000
11	1	14	170.58	2023-11-22 06:06:41.000	2023-11-22 14:23:27.000
12	2	1	251.94	2023-12-06 18:00:50.000	2023-12-07 00:57:20.000
13	2	2	260.11	2023-11-27 01:04:18.000	2023-11-27 01:22:38.000
14	2	2	351.21	2023-12-03 08:02:55.000	2023-12-03 08:21:01.000
15	2	2	777.24	2023-12-12 16:23:42.000	2023-12-12 19:05:02.000
16	2	2	874.11	2023-12-15 09:04:11.000	2023-12-15 13:41:53.000
17	2	2	343.14	2023-12-24 07:57:26.000	2023-12-24 15:33:46.000
18	2	2	323.47	2023-12-26 18:15:31.000	2023-12-26 22:12:37.000
19	2	2	578.28	2023-12-27 07:47:47.000	2023-12-27 09:35:16.000
20	2	2	200	2024-03-29 12:00:00.000	2024-03-29 13:00:00.000
21	3	3	453.63	2023-11-21 04:41:59.000	2023-11-21 05:53:30.000
22	3	3	989.94	2023-12-07 18:47:29.000	2023-12-08 00:18:21.000
23	3	3	6.53	2023-12-12 01:35:08.000	2023-12-12 03:36:30.000
24	3	3	112.89	2023-12-13 03:24:49.000	2023-12-13 11:17:08.000
25	3	3	731.39	2023-12-16 02:29:02.000	2023-12-16 09:37:43.000

Query executed successfully. 155.69,100.36 (16.0 RTM)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from SHOPPING;
```

100 %

	SID	UID	Amount_spent	Date_time_in	Date_time_out
25	3	3	731.39	2023-12-16 02:29:02.000	2023-12-16 09:37:43.000
26	3	3	629.77	2023-12-26 01:25:54.000	2023-12-26 05:05:00.000
27	3	3	382.63	2023-12-29 16:37:15.000	2023-12-29 23:07:12.000
28	3	3	906.83	2024-01-02 16:24:39.000	2024-01-02 16:25:26.000
29	3	3	250	2024-03-30 14:00:00.000	2024-03-30 15:00:00.000
30	3	13	837.92	2023-11-28 13:30:49.000	2023-11-28 17:33:04.000
31	4	1	300	2024-03-31 16:00:00.000	2024-03-31 17:00:00.000
32	4	20	514.29	2023-11-21 04:18:02.000	2023-11-21 05:54:25.000
33	5	2	350	2024-04-01 18:00:00.000	2024-04-01 19:00:00.000
34	5	10	921.45	2024-01-02 13:31:24.000	2024-01-02 18:55:19.000
35	6	3	400	2024-04-02 20:00:00.000	2024-04-02 21:00:00.000
36	6	14	653.34	2023-11-28 20:30:38.000	2023-11-29 01:11:37.000
37	8	10	716.01	2023-11-27 00:37:53.000	2023-11-27 05:45:02.000
38	8	18	11.55	2024-01-02 13:51:19.000	2024-01-02 17:08:20.000
39	9	4	891.04	2023-12-06 12:51:51.000	2023-12-06 18:10:47.000
40	9	19	767.46	2023-12-04 23:12:15.000	2023-12-05 07:15:30.000
41	10	7	818.65	2023-12-05 08:26:47.000	2023-12-05 17:15:19.000
42	15	14	364.86	2023-11-26 18:01:02.000	2023-11-26 23:22:14.000
43	15	20	130.05	2023-11-30 07:27:36.000	2023-11-30 15:32:38.000
44	17	18	70.52	2023-12-16 01:55:59.000	2023-12-16 07:49:36.000
45	17	19	874.6	2023-12-04 05:50:41.000	2023-12-04 06:28:44.000
46	18	20	443.65	2024-01-04 05:13:45.000	2024-01-04 08:23:18.000
47	19	8	610.71	2023-12-15 17:02:19.000	2023-12-15 21:36:09.000
48	20	11	719.66	2023-12-09 19:29:14.000	2023-12-09 22:51:03.000

DINING(UID, OID, Amount_spent, Date_time_in, Date_time_out)

SQLQuery12.sql - 1...p2g1 (rep2g1 (86))* SQLQuery11.sql - 1...2g1 (rep2g1 (269))*

Select * from DINING;

100 %

Results Messages

	UID	OID	Amount_spent	Date_time_in	Date_time_out
1	1	1	100	2024-03-28 10:00:00.000	2024-03-28 11:00:00.000
2	1	2	100	2024-03-31 16:00:00.000	2024-03-31 17:00:00.000
3	1	3	100	2024-03-29 12:00:00.000	2024-03-29 13:00:00.000
4	1	4	100	2024-04-01 18:00:00.000	2024-04-01 19:00:00.000
5	1	5	100	2024-03-30 14:00:00.000	2024-03-30 15:00:00.000
6	1	6	100	2024-04-02 20:00:00.000	2024-04-02 21:00:00.000
7	2	1	200	2024-03-28 10:00:00.000	2024-03-28 11:00:00.000
8	2	2	200	2024-03-31 16:00:00.000	2024-03-31 17:00:00.000
9	2	3	200	2024-03-29 12:00:00.000	2024-03-29 13:00:00.000
10	2	4	200	2024-04-01 18:00:00.000	2024-04-01 19:00:00.000
11	2	5	200	2024-03-30 14:00:00.000	2024-03-30 15:00:00.000
12	3	1	100	2024-03-28 10:00:00.000	2024-03-28 11:00:00.000
13	3	1	100	2024-03-29 12:00:00.000	2024-03-29 13:00:00.000
14	3	1	100	2024-03-30 14:00:00.000	2024-03-30 15:00:00.000
15	3	2	100	2024-03-31 16:00:00.000	2024-03-31 17:00:00.000
16	3	2	100	2024-04-01 18:00:00.000	2024-04-01 19:00:00.000
17	3	2	100	2024-04-02 20:00:00.000	2024-04-02 21:00:00.000
18	5	1	50	2024-03-31 12:00:00.000	2024-03-31 13:00:00.000
19	5	2	60	2024-03-31 19:00:00.000	2024-03-31 20:00:00.000
20	6	1	75	2024-03-31 12:00:00.000	2024-03-31 13:00:00.000
21	7	2	15	2024-02-15 19:00:00.000	2024-03-31 20:00:00.000
22	7	2	5	2024-03-16 11:00:00.000	2024-03-31 12:00:00.000
23	7	2	5	2024-03-17 11:00:00.000	2024-03-31 12:00:00.000
24	7	2	45	2024-03-31 19:00:00.000	2024-03-31 20:00:00.000
25	21	5	22.5	2024-04-01 10:00:00.000	2024-04-01 11:00:00.000
26	21	5	25	2024-04-01 12:00:00.000	2024-04-01 13:00:00.000
27	22	4	30	2024-04-01 12:00:00.000	2024-04-01 13:00:00.000
28	23	6	27.75	2024-04-01 12:00:00.000	2024-04-01 13:00:00.000
29	24	3	29.75	2024-04-01 12:00:00.000	2024-04-01 13:00:00.000

Query executed successfully.

155.69.100.36 (16.0 RTM) | rep2g1 (86) | rep2g1 | 00:00:00 | 29 rows

MALL(MID, Address, NumShops, CID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from MALL;
```

100 %

Results Messages

	MID	Address	NumShops	CID
1	1	789 Shopping Ave	100	1
2	2	101 Shopping Ave	150	2
3	3	202 Shopping Ave	120	1
4	4	Mall 4 Address	8	2
5	5	Mall 5 Address	9	3
6	6	Mall 6 Address	10	1
7	7	Mall 7 Address	11	4
8	8	Mall 8 Address	6	4
9	9	Mall 9 Address	12	5
10	10	Mall 10 Address	10	5

MALL_MGMT_COMPANY(CID, Address)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* - X

```
Select * from MALL_MGMT_COMPANY;
```

100 %

Results Messages

	CID	Address
1	1	123 Corporate Blvd
2	2	456 Corporate Blvd
3	3	Mall Management Company 3 Address
4	4	Mall Management Company 4 Address
5	5	Mall Management Company 5 Address

RESTAURANT_OUTLET(OID, Unit_no, MID, RID, Type, Name, Floor)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* - X

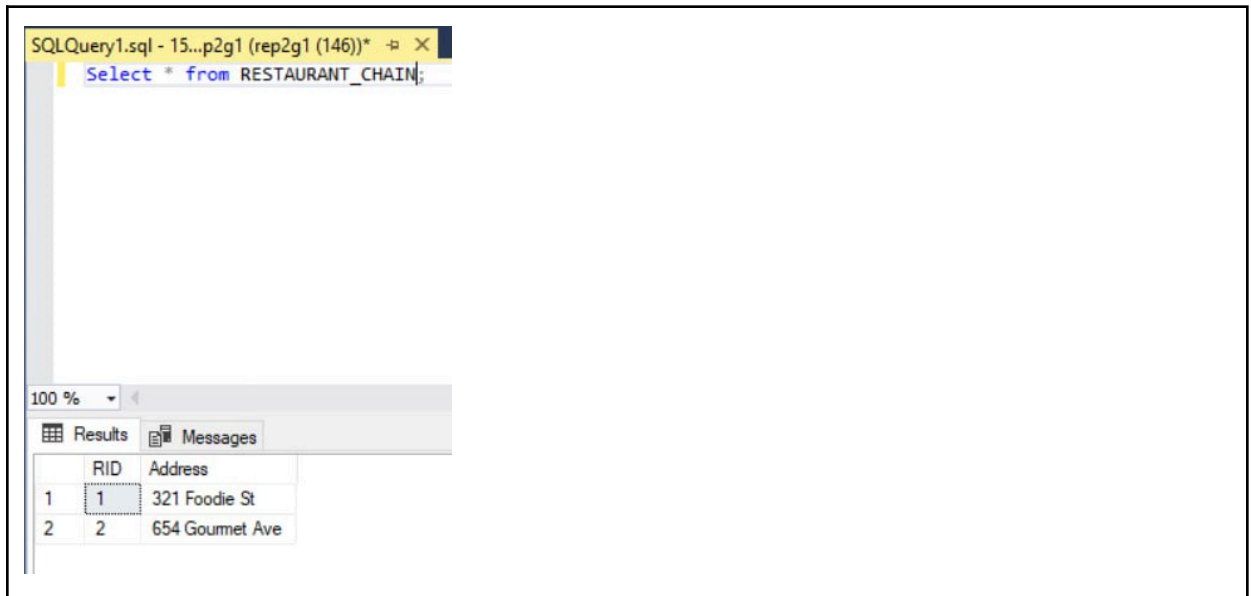
```
Select * from RESTAURANT_OUTLET;
```

100 %

Results Messages

	OID	Unit_no	MID	RID	Type	Name	Floor
1	1	Mall1 Unit 101	1	1	Fast Food	Burger Bonanza	1
2	2	Mall1 Unit 202	1	2	Cafe	Cafe Goodness	2
3	3	Mall2 Unit 103	2	2	Italian	Pasta Paradise	1
4	4	Mall2 Unit 204	2	1	Coffee Shop	Caffeine Dreams	2
5	5	Mall3 Unit 105	3	1	Coffee Shop	Caffeine Dreams	1
6	6	Mall3 Unit 206	3	2	Coffee Shop	Caffeine Dreams	2
7	7	Mall4 Unit 107	4	1	Coffee Shop	Caffeine Dreams	1
8	8	Mall4 Unit 208	4	2	Coffee Shop	Caffeine Dreams	2

RESTAURANT_CHAIN(RID, Address)



DAY_PACKAGE (DID, Description, VID, UID, Date, ProductID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))

Select * from DAY_PACKAGE;

100 %

Results

Messages

	DID	Description	VID	UID	Date	ProductID
1	1	Beach Day	1	1	2024-03-21 00:00:00.000	100
2	2	Beach Day	2	2	2024-03-21 00:00:00.000	100
3	3	Beach Day	3	3	2024-03-21 00:00:00.000	100
4	4	Beach Day	1	4	2024-03-21 00:00:00.000	100
5	5	Beach Day	2	5	2024-03-21 00:00:00.000	100
6	6	Beach Day	3	6	2024-03-21 00:00:00.000	100
7	7	Mountain Hike	1	19	2024-03-22 00:00:00.000	101
8	8	Mountain Hike	2	20	2024-03-22 00:00:00.000	101
9	9	Mountain Hike	3	21	2024-03-22 00:00:00.000	101
10	10	Mountain Hike	1	22	2024-03-22 00:00:00.000	101
11	11	Mountain Hike	2	23	2024-03-22 00:00:00.000	101
12	12	Mountain Hike	3	24	2024-03-22 00:00:00.000	101
13	13	Mountain Hike	1	25	2024-03-22 00:00:00.000	101
14	14	Mountain Hike	2	26	2024-03-22 00:00:00.000	101
15	15	Beach Day	3	11	2024-03-22 00:00:00.000	100
16	16	Beach Day	3	12	2024-03-22 00:00:00.000	100
17	17	Beach Day	2	13	2024-03-23 00:00:00.000	100
18	18	Beach Day	2	14	2024-03-24 00:00:00.000	100
19	19	Shopping Day	10	15	2024-03-25 00:00:00.000	103
20	20	Shopping Day	10	16	2024-03-25 00:00:00.000	103
21	21	Shopping Day	10	17	2024-03-25 00:00:00.000	103
22	22	Mall Day	5	5	2024-03-26 00:00:00.000	110
23	23	Mall Day	5	6	2024-03-26 00:00:00.000	110
24	24	Mall Day	5	7	2024-03-26 00:00:00.000	110
25	25	Mall Day	5	8	2024-03-26 00:00:00.000	110

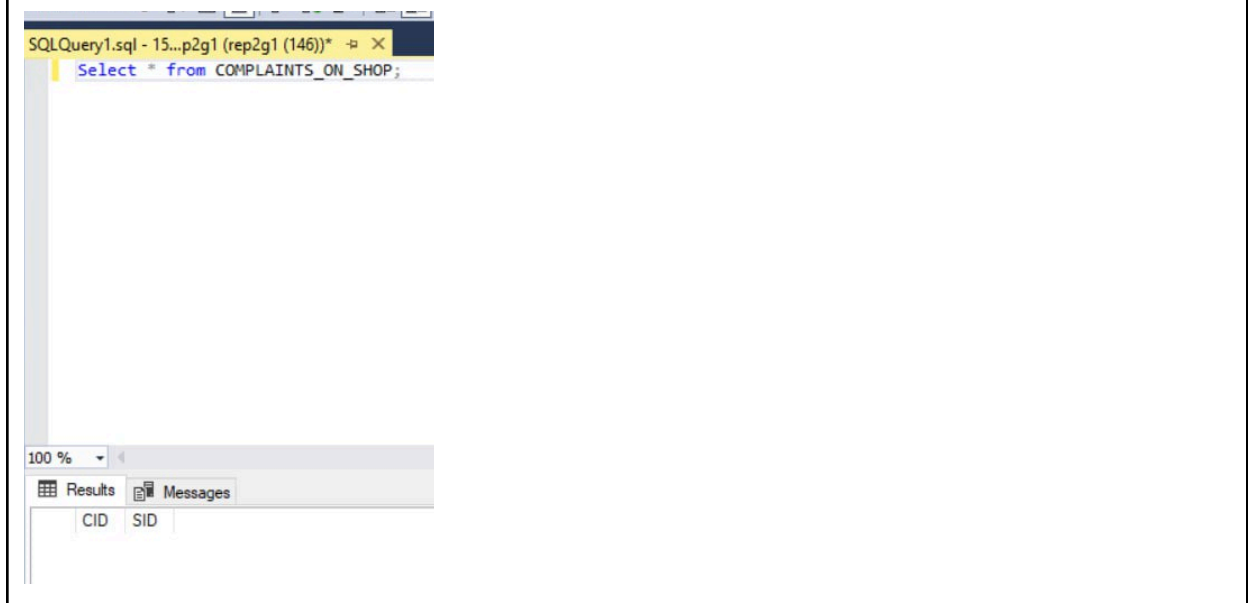
COMPLAINT (CID,Text,Status,Filed_date_time,UID)

The image shows a screenshot of a SQL query editor window. The title bar at the top reads "SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*". The main text area contains the SQL query: `Select * from COMPLAINT;`. Below the query area, there is a status bar showing "100 %". At the bottom, there are two tabs: "Results" and "Messages". The "Results" tab is active, and it displays a table with five columns: "CID", "Text", "Status", "Filed_date_time", and "UID".

COMPLAINTS_ON_RESTAURANT (CID, OID)



COMPLAINTS_ON_SHOP(CID, SID)



VOUCHER(VID, Date_issued, Description, Status, Expiry_date)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

Select * from VOUCHER;

100 %

Results Messages

	VID	Date_issued	Description	Status	Expiry_date
1	1	2024-03-01 00:00:00.000	Discount for Spa Treatment	Active	2024-04-01 00:00:00.000
2	2	2024-03-02 00:00:00.000	Free Meal at Restaurant	Active	2024-04-02 00:00:00.000
3	3	2024-03-03 00:00:00.000	50% Off on Adventure Sports	Active	2024-04-03 00:00:00.000
4	4	2024-03-04 00:00:00.000	Free Night Stay at Resort	Active	2024-04-04 00:00:00.000
5	5	2024-03-05 00:00:00.000	Complimentary Breakfast Buffet	Active	2024-04-05 00:00:00.000
6	6	2024-03-06 00:00:00.000	Spa Package for Two	Active	2024-04-06 00:00:00.000
7	7	2024-03-07 00:00:00.000	Scuba Diving Lesson	Active	2024-04-07 00:00:00.000
8	8	2024-03-08 00:00:00.000	Discount on Local Sightseeing Tour	Active	2024-04-08 00:00:00.000
9	9	2024-03-09 00:00:00.000	Free Upgrade to Luxury Suite	Active	2024-04-09 00:00:00.000
10	10	2024-03-10 00:00:00.000	50% Off on Dinner Cruise	Active	2024-04-10 00:00:00.000

PURCHASE_VOUCHER(VID, Purchase_discount, UID, Date_time)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

Select * from PURCHASE_VOUCHER;

100 %

Results Messages

	VID	Purchase_discount	UID	Date_time
1	2	5	1	2024-03-26 00:00:00.000

DINE_VOUCHER(VID, Cash_discount, UID, Date_time)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* -p X

```
Select * from DINE_VOUCHER;
```

100 %

Results					Messages				
	VID	Cash_discount	UID	Date_time					
1	1	10	1	2024-03-25 00:00:00.000					

GROUP_VOUCHER(VID, Group_size, Group_discount, UID, Date_time)

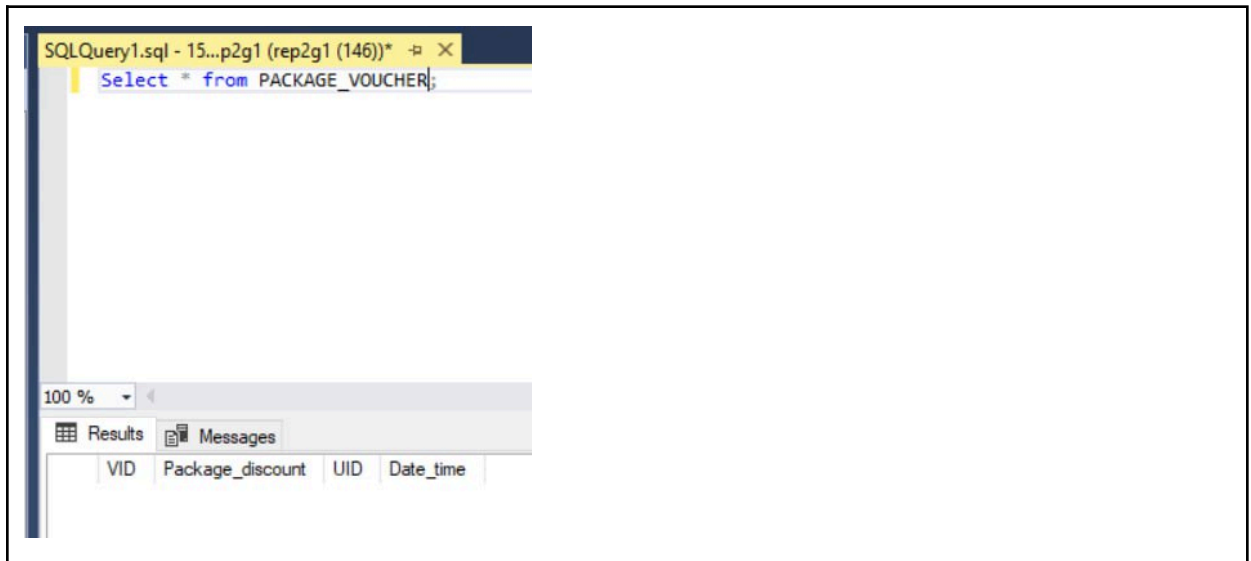
SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* -p X

```
Select * from GROUP_VOUCHER;
```

100 %

Results					Messages				
	VID	Group_size	Group_discount	UID	Date_time				

PACKAGE_VOUCHER(VID, Package_discount, UID, Date_time)



RECOMMENDATION(NID, Valid_period, Date_issued, MID, Dine_voucher, Purchase_voucher, DID, OID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from RECOMMENDATION;
```

100 %

Results Messages

	NID	Valid_period	Date_issued	MID	Dine_voucher	Purchase_voucher	DID	OID
1	1	30	2024-03-28 00:00:00.000	1	1	2	1	1
2	2	30	2024-03-29 00:00:00.000	1	1	2	1	2
3	3	30	2024-03-30 00:00:00.000	2	1	2	1	1
4	4	30	2024-03-31 00:00:00.000	2	1	2	1	2
5	5	30	2024-04-01 00:00:00.000	3	1	2	1	1
6	6	30	2024-03-31 00:00:00.000	1	1	2	1	1

USER_RELATED_USER(UID1, UID2, Type, GID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* -p X

```
Select * from USER_RELATED_USER;
```

100 %

Results				
	UID1	UID2	Type	GID
1	1	2	family	11
2	2	3	family	11
3	3	4	family	11
4	4	5	family	1
5	5	6	family	1
6	6	7	family	1
7	7	8	family	22
8	8	9	family	22
9	9	10	family	22
10	11	12	club	2
11	12	13	club	2
12	13	14	club	2
13	14	15	club	2
14	15	16	club	2
15	16	17	club	2
16	17	18	club	2
17	18	19	club	2
18	19	20	club	2
19	20	11	club	2
20	21	22	friends	3
21	22	23	friends	3
22	23	24	friends	3
23	24	25	friends	3
24	25	26	friends	3
25	26	27	friends	3

25	26	27	friends	3
26	27	28	friends	3
27	28	29	friends	3
28	29	30	friends	3
29	30	21	friends	3

USER_USE_RECOMMENDATION(UID, NID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from USER_USE_RECOMMENDATION;
```

100 %

Results Messages

UID	NID
-----	-----

DAY_PACKAGE_HAS_MALL(DID, MID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))*

```
Select * from DAY_PACKAGE_HAS_MAL
```

100 %

Results Messages

DID	MID
-----	-----

DAY_PACKAGE_HAS_RESTAURANT_OUTLET(DID, OID)

SQLQuery1.sql - 15...p2g1 (rep2g1 (146))* - [X]

```
Select * from DAY_PACKAGE_HAS_RESTAURANT_OUTLET;
```

100 %



Results



Messages

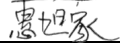
DID

OID

APPENDIX C: INDIVIDUAL CONTRIBUTION FORM

Full Name	Individual Contribution to Lab 1 Submission	Percentage of Contribution	Signature

Full Name	Individual Contribution to Lab 3 Submission	Percentage of Contribution	Signature

Full Name	Individual Contribution to Lab 5 Submission	Percentage of Contribution	Signature
HUI XUJIA	Inserting Sample Data, Queries for 1,2,4,5,7. Brief explanation for 1,2,3,4,5,6,7	25%	
LIM ZHI YONG	Creating and Populating Tables, Queries for 1,2,4,5,7. Brief explanation for 1,2,3,4,5,6,7	25%	
ETHAN LEE KEI SHEN	Populated some tables and a little bit of query 1,3 and 4.	12.5%	
HUANG CAIHONG	Creating and populating tables. Queries 3,6 and 7. Brief explanation for 3 and 6.	12.5%	
CHONG HUAI ZHI	Creating Tables, filling up documents for printout for all table records and query outputs, brief explanation for 4.	12.5%	
BHATI NANCY	Queries for 3 and 6. Screenshots of outputs in the PDF and the video recording of all queries.	12.5%	Nancy

APPENDIX D: USE OF AI TOOL(S) IN LAB WORK

Each team member should indicate either A or B:

A. I affirm that my contribution(s) to the lab work is my own, produced without help from any AI tool(s).

B. I affirm that my contribution(s) to the lab work has been produced with the use of AI tool(s).

Team member (full name)	Signature	Date	A or B
HUI XUJIA		31/03/24	A
LIM ZHI YONG		31/03/24	A
ETHAN LEE KEI SHEN		31/03/24	A
HUANG CAIHONG		31/03/24	A
CHONG HUAI ZHI		31/03/24	A
BHATI NANCY	Nancy	31/03/24	A

By signing this form, you declare that the above affirmation made is true and that you have read and understood NTU's policy on the use of AI tools.

If any team member answered B, the team member(s) must indicate and replicate the table below for every instance AI tool(s) is used:

Name of AI tool	< For example, ChatGPT >
Input prompt	< Insert the question that you asked ChatGPT >
Date generated	
Output generated	< Insert the response verbatim from ChatGPT >
Output screenshots	
Impact on submission	< Briefly explain which part of your submitted work was ChatGPT's response applied >